



UNIVERSIDADE ESTADUAL DE RORAIMA

Disciplina: Sistemas Multimídia

Docente: Caio de Jesus Gregoratto

Acadêmico: Aldo Elysio Campos Barbosa

Relatório de Projeto Final

1.Introdução:

Este relatório detalha o processo de desenvolvimento de um protótipo de jogo 2D do gênero RPG de Turno, intitulado "My Final Stand". O projeto foi desenvolvido na linguagem C++ utilizando a biblioteca SFML 3.0.0, e cumpre todos os requisitos solicitados na especificação do trabalho, incluindo o gerenciamento de duas cenas distintas (Menu e Combate) e a implementação de uma lógica de turnos.

2.Requisitos:

1. Carregamento de texturas;

2. Exibição de Sprites;
3. Definição de Shapes (círculos, retângulos etc.);
4. Exibição de Shapes;
5. Definição de textos;
6. Exibição de textos;
7. Leitura de inputs;
8. Interação baseadas nos inputs;
9. Lógica (Parar, iniciar, mudar, etc.);
10. No mínimo 02 (duas) cenas.

3. Ferramentas e Arquitetura

Máquina de Estados Finitos (FSM): A lógica principal do jogo é controlada por um enum *GameState* {menu, combat, gameover}. O loop principal do jogo (no main.cpp) utiliza um switch (*currentState*) para decidir qual lógica de input, atualização e renderização deve ser executada, separando completamente as "cenas".

Programação Orientada a Objetos (OOP): Foram criadas classes personalizadas para encapsular responsabilidades, notavelmente:

- **Animator:** Classe que herda de *sf::Sprite* e gerencia o "pré-carregamento" de texturas (PNGs) e a lógica de animação (o seu "protocolo" de loop, one-tap, etc.).
- **Entity:** Classe que "possui" um *Animator* e adiciona lógica de jogo, como hp (pontos de vida).

4. Cumprimento de requisitos

1. Carregamento de Texturas:

As texturas (arquivos .png) são carregadas do disco usando *sf::Texture::loadFromFile()*. Para evitar erros de caminho, foi

utilizada uma função auxiliar *getPath()* que constrói o caminho absoluto para a pasta *assets*, garantindo que as texturas sejam encontradas (ex: *assetsPath + "enemy_1.png"*).

2. Exibição de Sprites:

A exibição de sprites é gerenciada pela classe *Animator* que herda de *sf::Sprite*, ela desenha os sprites diretamente na janela com *window.draw(player1.animator)*.

3. Definição de Shapes:

Shapes foram usados para a Interface do Usuário (UI). A barra de HP do inimigo(*enemy_hp*) e o contorno do título no menu (*rectangle*) foram definidos usando *sf::RectangleShape*.

4. Exibição de Shapes:

A barra de HP é desenhada na tela durante o estado *GameState::combat* usando *window.draw(player_hp)*. A lógica de atualização da barra (*enemy_hp.setSize(...)*) é um exemplo de manipulação de Shape em tempo real.

5. Definição de Textos:

Os textos foram definidos carregando fontes (.ttf) com *sf::Font::openFromFile()* e aplicando-as a objetos *sf::Text*.

6. Exibição de Textos:

Os textos são desenhados na janela com base no estado atual. No case *GameState::menu*;, a tela desenha *window.draw(text_title)* e *window.draw(text_press)*.

7. Leitura de Inputs:

A leitura de inputs é feita no loop de eventos principal, usando o novo padrão do SFML 3.0: *while (auto event = window.pollEvent())* que gerencia os inputs de “Z”, “G” e “Esc”.

8. Interação Baseada nos Inputs:

A lógica de interação é gerenciada pelo switch (*currentState*). Isso cumpre o requisito de inputs diferentes por cena:

- **Cena menu:** A tecla '**Z**' é usada para *currentState = GameState::combat*; (iniciar o jogo).
- **Cena combat:** A tecla '**Z**' é usada para *currentTurn = turn::playerAction*; (iniciar o ataque).
- **Global:** A tecla '**ESC**' é usada para *window.close()* em qualquer cena.

9. Lógica (Parar, Iniciar, Mudar):

A lógica é aplicada na máquina de estados de turno dentro do combate, controlada pelo enum *turn*. O estado se inicia com *turn::playerTurn* (turno que o jogador escolhe uma ação) seguida por *turn::playerAction* (turno em que a ação e sua respectiva animação é executada), a lógica chama *Animator* para checar se a animação terminou (*!player1.animator.isPlaying()*), para parar a ação e mudar o turno (*currentTurn = turn::enemyTurn*). Se o hp de nenhuma entidade é menor que zero, o turno segue para o turno inimigo, um delay de aproximadamente 1seg acontece para simular a IA do inimigo, logo em seguida o inimigo ataca, a mesma lógica de animação é aplicada a ele, o HP das entidades são verificados novamente e o turno volta para o jogador, reiniciando o loop da lógica. Se em um dos turnos de ação (do jogador ou do inimigo) a condição do HP for atendida o turno passa a ser *turn::finished* que ativa a cena de Vitória do jogador.

10. No Mínimo 02 (duas) Cenas:

O projeto implementa 3 cenas principais através do enum *GameState*: menu, combat e gameover. O switch principal garante que os recursos de 1 a 9 estejam presentes e funcionem de forma independente em cada cena.

5.Instruções

Compilação: O projeto foi configurado para ser compilado no Visual Studio Code com o compilador MinGW (g++). O arquivo .vscode/tasks.json (incluso no anexo) contém todas as configurações de build necessárias. Ele foi configurado para:

1. Compilar main.cpp e todos os arquivos da pasta /src.
2. Buscar os cabeçalhos (.hpp) na pasta /include.
3. Linkar estaticamente com as bibliotecas do SFML 3.0.0 (-lsfml-graphics, -lsfml-window, -lsfml-system), na ordem correta de dependência.

Execução:

1. Execute o main.exe gerado.
2. **Nota:** Para a execução, os arquivos .dll do SFML (ex: sfml-graphics-3.dll, etc.) devem estar na mesma pasta do main.exe, ou o caminho para a pasta SFML-3.0.0/bin deve estar no PATH do sistema.

Controles do Jogo:

- **“Z”:** Pressione a tecla para iniciar a cena de combate (cena menu) e executar a ação de ataque (cena combate).
- **“G”:** Pressione a tecla para forçar o Gameover.
- **“Esc”:** Pressione a tecla para fechar o jogo.

6.Conclusão

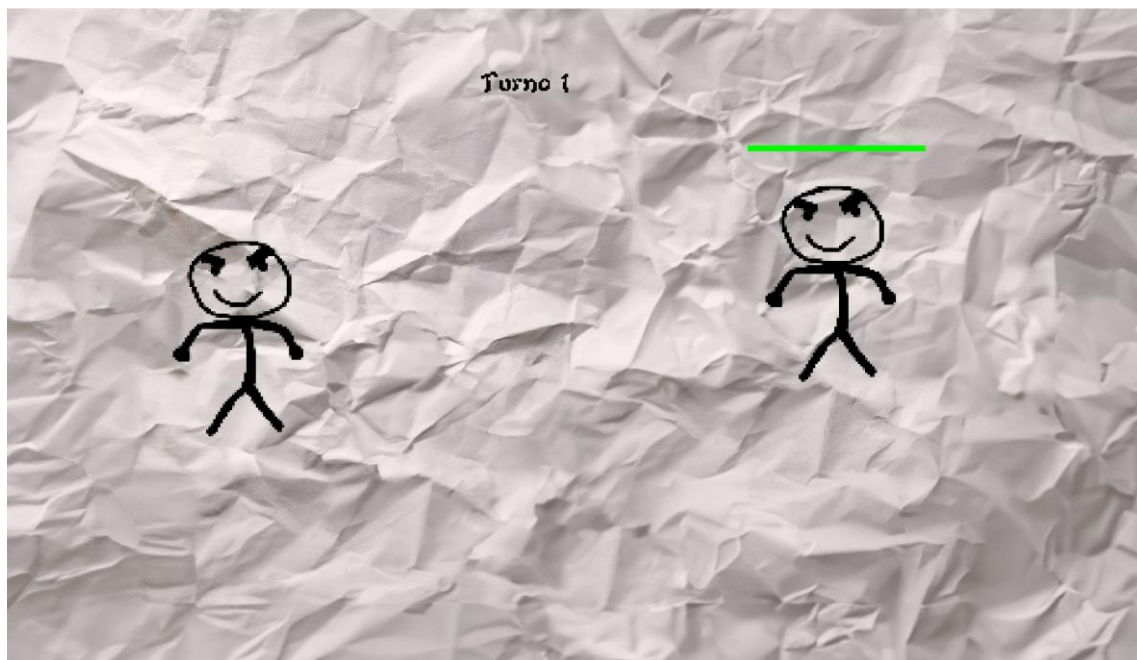
O desenvolvimento deste protótipo permitiu a aplicação prática de conceitos fundamentais de sistemas multimídia, desde o carregamento de assets (texturas, fontes) até a implementação de uma lógica de jogo complexa. O uso de uma Máquina de Estados Finitos (FSM) no main provou ser uma arquitetura eficaz para gerenciar diferentes cenas, enquanto a Programação Orientada a Objetos (classes Entity e Animator) permitiu encapsular a lógica de animação e

combate de forma limpa e escalável. O projeto final cumpre todos os 10 requisitos obrigatórios da disciplina.

7.Anexos



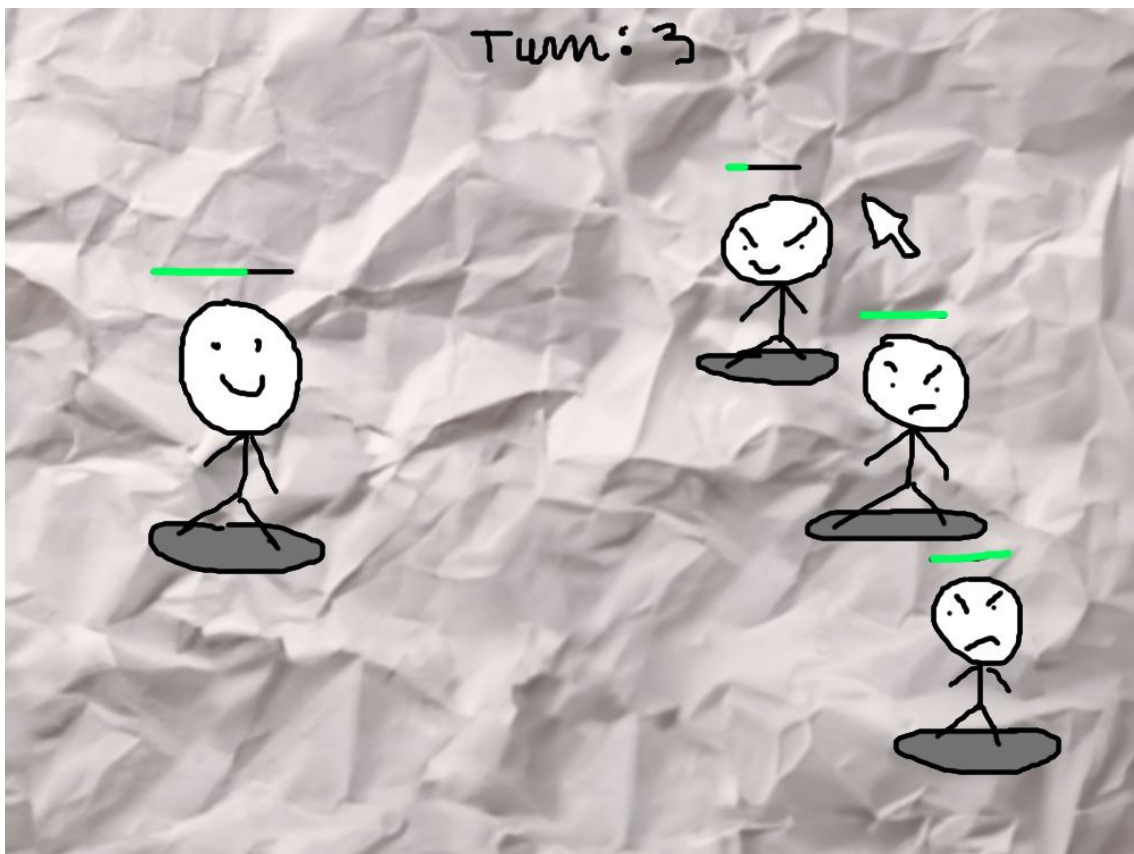
Cena Menu



Cena Combate



Cena Game Over



(Bônus) Arte conceitual